

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_191a9b86-991\agent-a5635e959407008a7.jsonl`
- SHA-256: `c0afa8643ecd65ccd42d11d2a9613d5ad23973529c133dca05860c5d9fa96d2e`
- Source modified: `2026-06-22T03:49:51+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_191a9b86-991`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T03:48:13.047Z)

Adversarially verify this P2c review finding by reading the cited code in C:\\Users\\avidu\\Projects\\khelsutra-p2c yourself (run `cd C:\\Users\\avidu\\Projects\\khelsutra-p2c && git diff origin/master...HEAD`). Is it a REAL defect to fix, or a false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: wizard-logic

Severity(claimed): nit

Title: KeyStep and ComputeStep can present an internally inconsistent state when default_segmenter is "gemini" but no key is present

File: web/src/components/SetupWizard.tsx:26,54

Detail: KeyStep keys off `caps?.gemini\_key\_present` and ComputeStep independently renders `caps?.default\_segmenter ?? "motion"`. If the engine reports `{gemini\_key\_present:false, default\_segmenter:"gemini"}` (e.g. a misconfigured box where gemini is the configured default but the key isn't readable), step 1 says "No AI key yet ? KhelSutra works offline with the built-in motion detector" while step 2 says "Detector: gemini". The two steps then contradict each other. This is the engine's reported state surfaced verbatim, so it's low-risk, but the wizard makes no attempt to reconcile or caveat the mismatch.

Fix: Optional: in ComputeStep, if `default\_segmenter==="gemini" && gemini\_key\_present!==true`, show the effective offline fallback (motion) or a short caveat, so the two steps don't contradict.

2. assistant (2026-06-22T03:48:14.880Z)

I'll verify this finding by examining the actual code in the cited file.

3. user (2026-06-22T03:48:18.560Z)

b67e758 chore(site): mirror wizard.* catalog keys into site/public/locales (copy:locales)
e6b9725 feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
cf59fbf feat(site): add es/da/id to the marketing site (parity with the SPA's 6 languages) (#87)

---DIFF STAT---

locales/da/common.json	32 ++++++
locales/en/common.json	32 ++++++
locales/es/common.json	32 ++++++
locales/hi/common.json	32 ++++++
locales/id/common.json	32 ++++++
locales/kn/common.json	32 ++++++
site/public/locales/da/common.json	32 ++++++
site/public/locales/en/common.json	32 ++++++
site/public/locales/es/common.json	32 ++++++
site/public/locales/hi/common.json	32 ++++++
site/public/locales/id/common.json	32 ++++++
site/public/locales/kn/common.json	32 ++++++
web/src/App.tsx	9 +-

```

web/src/api/client.ts | 6 ++
web/src/api/types.ts | 17 +++
web/src/components/SetupWizard.test.tsx | 115 ++++++
web/src/components/SetupWizard.tsx | 179 ++++++
web/src/components/a4.touch.test.tsx | 4 +
18 files changed, 713 insertions(+), 1 deletion(-)

```

4. assistant (2026-06-22T03:48:20.587Z)

Let me read the full SetupWizard component to verify the cited lines.

5. user (2026-06-22T03:48:21.707Z)

```

1      // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2      // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box
can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "kheIsutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15         try {
16             return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17         } catch {
18             return false; // private-mode / no storage ? show it (harmless; user can skip)
19         }
20     }
21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-block text-sm font-medium text-green underline"
41                         >
42                             {t("wizard.key.getKey")}
43                         </a>
44                     </>
45                 )}
46             </div>
47         );

```

```

48     }
49
50     function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51         const { t } = useTranslation();
52         const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53         const gpu = caps?.gpu?.nvidia_smi_present === true;
54         const detector = caps?.default_segmenter ?? "motion";
55         return (
56             <div>
57                 <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58                 <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59                 <ul className="mt-3 space-y-1 text-sm text-ink/70">
60                     <li>{t("wizard.compute.detector", { name: detector })}</li>
61                     <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62                     <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
63                 </ul>
64             </div>
65         );
66     }
67
68     function VideosStep() {
69         const { t } = useTranslation();
70         return (
71             <div>
72                 <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73                 <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74                 <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75             </div>
76         );
77     }
78
79     export function SetupWizard({ onClose }: { onClose: () => void }) {
80         const { t } = useTranslation();
81         const [step, setStep] = useState(0);
82         const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83         const [loading, setLoading] = useState(true);
84         const [offline, setOffline] = useState(false);
85
86         useEffect(() => {
87             let alive = true;
88             const cid = newCorrelationId();
89             getCapabilities(cid)
90                 .then((c) => {
91                     if (!alive) return;
92                     setCaps(c);
93                     setLoading(false);
94                 })
95                 .catch((err) => {
96                     if (!alive) return;
97                     // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                     setOffline(true);
99                     setLoading(false);
100                     log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101                 });
102             return () => {
103                 alive = false;
104             };
105         }, []);
106
107         const finish = () => {
108             try {
109                 localStorage.setItem(WIZARD_DONE_KEY, "true");
110             } catch {
111                 /* private mode: still close (the wizard just reappears next visit) */

```

```

112     }
113     onClose();
114 };
115
116 const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117 const isLast = step === STEPS - 1;
118
119 return (
120   <section
121     role="dialog"
122     aria-label={t("wizard.title")}
123     className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"
124   >
125     <div className="flex items-center justify-between gap-3">
126       <h2 className="text-xl font-bold">{t("wizard.title")}</h2>
127       <button
128         type="button"
129         onClick={finish}
130         className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131       >
132         {t("wizard.skip")}
133       </button>
134     </div>
135     <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136     {stepping && (
137       <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138     )}
139
140     <div className="mt-5 min-h-[128px]" aria-live="polite">
141       {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142       {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143       {stepping && step === 0 && <KeyStep caps={caps} />}
144       {stepping && step === 1 && <ComputeStep caps={caps} />}
145       {stepping && step === 2 && <VideosStep />}
146     </div>
147
148     <div className="mt-6 flex items-center justify-between">
149       {stepping ? (
150         <>
151           <button
152             type="button"
153             disabled={step === 0}
154             onClick={() => setStep((s) => Math.max(0, s - 1))}
155             className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156           >
157             {t("wizard.back")}
158           </button>
159           <button
160             type="button"
161             onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
162             1)))}
163             className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
164           >
165             {isLast ? t("wizard.done") : t("wizard.next")}
166           </button>
167         </>
168       ) : (
169         <button
170           type="button"
171           onClick={finish}
172           className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-

```

```

white"
172         >
173         {t("wizard.done")}
174     </button>
175     )}
176 </div>
177 </section>
178 );
179 }
180

```

6. assistant (2026-06-22T03:48:24.027Z)

The cited lines are confirmed. Now let me check the type definitions for `EngineCapabilities` to understand what `gemini\_key\_present` and `default\_segmenter` actually represent, and what the engine reports.

7. user (2026-06-22T03:48:25.140Z)

```

1 // Types mirror the engine's verified contract (badminton-highlight-indexer).
2 // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3 // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5 export interface RallyMetadata {
6     /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7     shot_count?: number;
8     /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9     shot_count_confidence?: string;
10    detector?: string;
11    [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;
19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34     status: string;
35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 export interface EngineConfig {
40     stitching?: {
41         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
42         min_shot_count?: number;
43         [key: string]: unknown;

```

```

44     };
45     [key: string]: unknown;
46 }
47
48 /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
49 * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-bools only (never the Gemini
key value).
50 * Every field optional/defensive: a missing field degrades the wizard gracefully. */
51 export interface EngineCapabilities {
52     api_version?: number;
53     engine_version?: string;
54     segmenters?: string[];
55     default_segmenter?: string;
56     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
57     gpu?: { nvidia_smi_present?: boolean };
58     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
59     gemini_key_present?: boolean;
60     wasb_weights_present?: boolean;
61     deployment?: { profile?: string; compute_target?: string };
62     [key: string]: unknown;
63 }
64
65 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
66 export interface ProcessResponse {
67     job_id: string;
68     /** Always "queued" at submit. */
69     status: string;
70     /** Relative poll path, e.g. "/api/jobs/<id>". */
71     poll: string;
72 }
73
74 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
75 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
76 export interface UploadInitResponse {
77     upload_id: string;
78     /** Per-part ceiling in MB ? slice the file into parts this big. */
79     max_part_mb: number;
80     next_part: string;
81 }
82 export interface UploadPartResponse {
83     upload_id: string;
84     received_parts: number;
85     received_bytes: number;
86 }
87 export interface UploadCompleteResponse {
88     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
89     path: string;
90     video_id: string;
91     size_bytes: number;
92 }
93
94 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
95 * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
96 * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
shaped
97 * FAILURE ? the UI must not treat it as a load. */
98 export interface JobStatus {
99     job_id: string;

```

```

100     status: "queued" | "running" | "done" | "failed";
101     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
102     progress?: string | null;
103     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
104     video_id?: string | null;
105     /** Execution error (worker crash / unexpected) when status === "failed". */
106     error?: string | null;
107     result?: { status?: string; video_id?: string; message?: string } | null;
108 }
109

```

8. assistant (2026-06-22T03:48:28.479Z)

Now let me understand the engine's actual behavior. The key question for the adversarial verification: can the engine actually report `{gemini\_key\_present: false, default\_segmenter: "gemini"}`? Let me look at the engine's `/api/capabilities` endpoint in the badminton-highlight-indexer repo to see how `default\_segmenter` is derived.

9. user (2026-06-22T03:48:31.255Z)

<persisted-output>

Output too large (29.2KB). Full output saved to: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\tool-results\toolu_01633svhN4R96Z4btTLiYnn2.txt

Preview (first 2KB):

```

artifacts\pr245_review.json-7-      "path": "docs/RALLY_DETECTION_CODE_REVIEW.md",
artifacts\pr245_review.json-8-      "line": 13,
artifacts\pr245_review.json-9-      "side": "RIGHT",
artifacts\pr245_review.json:10:[Omitted long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\pr245_review.json-11-    },
C:\Users\avidu\Projects\badminton-highlight-indexer\artifacts\pr245_review.json-12-    {
artifacts\pr245_review.json-13-      "path": "docs/RALLY_DETECTION_CODE_REVIEW.md",
--
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-125-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-126-
backend\config\models.py-127-class IndexingConfig(BaseModel):
backend\config\models.py:128:     default_segmenter: str = "yolo_hybrid"
backend\config\models.py-129-     use_gpu: bool = True
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-130-     # Detector
backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-131-     # WSL
runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
--
backend\config\models.py-207-     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
backend\config\models.py-208-     fusion: FusionConfig = Field(default_factory=FusionConfig)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-209-
backend\config\models.py:210:     @field_validator("default_segmenter")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-211-
@classmethod
backend\config\models.py-212-     def segmenter_must_be_registered(cls, v: str) -> str:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py-213-     #
Registry check happens at runtime in main/segmenter selection.
--
backend\worker\__main__.py-196-     print("[worker] validation FAILED: "
...
</persisted-output>

```

10. assistant (2026-06-22T03:48:33.439Z)

Let me find the actual `/api/capabilities` endpoint implementation to see how `default\_segmenter` is populated in the response.

11. user (2026-06-22T03:48:34.204Z)

No files found

12. user (2026-06-22T03:48:37.250Z)

No files found

13. assistant (2026-06-22T03:48:41.627Z)

The capabilities endpoint may not yet exist in the engine (or is on a different branch). Let me search more broadly.

14. user (2026-06-22T03:48:41.951Z)

No files found

15. assistant (2026-06-22T03:48:45.355Z)

The engine endpoint isn't present in the local checkout of badminton-highlight-indexer. The wizard is built defensively against a contract. Let me check the SetupWizard test to see what scenarios are explicitly tested, and whether the contradiction case is acknowledged.

16. user (2026-06-22T03:48:46.179Z)

```
1      // @vitest-environment jsdom
2      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
3      import { render, screen, fireEvent, cleanup, waitFor } from "@testing-library/react";
4      import i18n from "../i18n/index";
5      import { SetupWizard, WIZARD_DONE_KEY, isWizardDone } from "../SetupWizard";
6      import type { EngineCapabilities } from "../api/types";
7
8      function stubCapabilities(caps: EngineCapabilities | "reject") {
9          vi.stubGlobal(
10              "fetch",
11              vi.fn(async (url: string | URL) => {
12                  const u = typeof url === "string" ? url : url.toString();
13                  if (u.endsWith("/api/capabilities")) {
14                      if (caps === "reject") throw new TypeError("Failed to fetch");
15                      return { ok: true, status: 200, json: async () => caps } as unknown as Response;
16                  }
17                  return { ok: false, status: 404, text: async () => "" } as unknown as Response;
18              }),
19          );
20      }
21
22      const LIGHT: EngineCapabilities = {
23          gemini_key_present: false,
24          default_segmenter: "motion",
25          ffmpeg: { ffmpeg: false, ffprobe: false },
26          gpu: { nvidia_smi_present: false },
27      };
28      const READY: EngineCapabilities = {
29          gemini_key_present: true,
30          default_segmenter: "gemini",
31          ffmpeg: { ffmpeg: true, ffprobe: true },
32          gpu: { nvidia_smi_present: true },
33      };
34
35      beforeEach(async () => {
36          window.localStorage.clear();
37          await i18n.changeLanguage("en");
38      });
39      afterEach(() => {
```

```

40     cleanup();
41     vi.restoreAllMocks();
42     vi.unstubAllGlobals();
43 });
44
45 describe("SetupWizard", () => {
46     it("checks capabilities, then step 1 guides a missing Gemini key (offline-first
default)", async () => {
47         stubCapabilities(LIGHT);
48         render(<SetupWizard onClose={() => {}} />);
49         expect(screen.getByText(/Checking what this device can do/)).toBeTruthy();
50         expect(await screen.findByText(/built-in motion detector/)).toBeTruthy();
51         expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
52         expect(screen.getByText(/Step 1 of 3/)).toBeTruthy();
53     });
54
55     it("walks all 3 steps from real capabilities; Back returns; Done sets the flag +
closes", async () => {
56         stubCapabilities(LIGHT);
57         const onClose = vi.fn();
58         render(<SetupWizard onClose={onClose} />);
59         await screen.findByText(/built-in motion detector/);
60
61         fireEvent.click(screen.getByText("Next")); // ? compute
62         expect(screen.getByText("Detector: motion")).toBeTruthy();
63         expect(screen.getByText(/ffmpeg not found/)).toBeTruthy();
64         expect(screen.getByText(/No GPU needed/)).toBeTruthy();
65
66         fireEvent.click(screen.getByText("Back")); // ? key
67         expect(screen.getByText("Get a free Gemini key")).toBeTruthy();
68
69         fireEvent.click(screen.getByText("Next")); // ? compute
70         fireEvent.click(screen.getByText("Next")); // ? videos
71         expect(screen.getByText(/You're all set/)).toBeTruthy();
72
73         fireEvent.click(screen.getByText("Get started")); // Done on last step
74         expect(onClose).toHaveBeenCalledTimes(1);
75         expect(window.localStorage.getItem(WIZARD_DONE_KEY)).toBe("true");
76     });
77
78     it("reflects a connected key + a GPU/ffmpeg-ready box", async () => {
79         stubCapabilities(READY);
80         render(<SetupWizard onClose={() => {}} />);
81         expect(await screen.findByText(/Gemini is connected/)).toBeTruthy();
82         fireEvent.click(screen.getByText("Next"));
83         expect(screen.getByText("Detector: gemini")).toBeTruthy();
84         expect(screen.getByText(/Video tools \(\ffmpeg\) ready/)).toBeTruthy();
85         expect(screen.getByText(/A GPU was detected/)).toBeTruthy();
86     });
87
88     it("Skip closes + sets the flag without finishing the steps", async () => {
89         stubCapabilities(LIGHT);
90         const onClose = vi.fn();
91         render(<SetupWizard onClose={onClose} />);
92         await screen.findByText(/built-in motion detector/);
93         fireEvent.click(screen.getByText("Skip"));
94         expect(onClose).toHaveBeenCalledTimes(1);
95         expect(isWizardDone()).toBe(true);
96     });
97
98     it("degrades gracefully when the engine is unreachable (offline) ? message + a Done
escape", async () => {
99         stubCapabilities("reject");
100        const onClose = vi.fn();
101        render(<SetupWizard onClose={onClose} />);

```

```

102     expect(await screen.findByText(/Couldn't reach KhelSutra/)).toBeTruthy();
103     // No step content / step indicator when offline.
104     expect(screen.queryByText(/Step 1 of 3/)).toBeNull();
105     fireEvent.click(screen.getByText("Get started"));
106     await waitFor(() => expect(onClose).toHaveBeenCalled());
107     expect(isWizardDone()).toBe(true);
108   });
109
110   it("isWizardDone reads the localStorage flag", () => {
111     expect(isWizardDone()).toBe(false);
112     window.localStorage.setItem(WIZARD_DONE_KEY, "true");
113     expect(isWizardDone()).toBe(true);
114   });
115 });
116

```

17. assistant (2026-06-22T03:48:51.258Z)

The tests confirm the two scenarios are coupled in test fixtures: `LIGHT` (key=false, segmenter=motion) and `READY` (key=true, segmenter=gemini). The finding hinges on whether the engine can produce the cross product `{gemini\_key\_present: false, default\_segmenter: "gemini"}`.

Let me verify how the engine actually resolves `default\_segmenter` when there's no key. The critical question: does the engine fall back the *configured* default to "motion" when the key is missing, or does it report the configured value verbatim? Let me check the engine's config resolution / segmenter selection logic in the badminton repo.

18. user (2026-06-22T03:48:53.205Z)

```

6fdd1d4 feat(serving): exposure-safety boot posture ? fail fast on a half-configured exposed
engine (default-OFF)
serving/exposure-safety-preflight
---
backend/config/models.py:128:     default_segmenter: str = "yolo_hybrid"
backend/config/models.py:210:     @field_validator("default_segmenter")
backend/main.py:173:         "default_segmenter": getattr(indexing, "default_segmenter", None),
backend/main.py:206:     default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
backend/main.py:716:         seg_name = args.segmenter or getattr(getattr(global_config,
"indexing", None), "default_segmenter", "motion")
backend/reporting/harvest.py:216:     config_default = (config.get("indexing") or
{}).get("default_segmenter")
backend/worker/__main__.py:199:         seg_name = args.segmenter or
config.indexing.default_segmenter
backend/worker/__main__.py:459:     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")

```

19. assistant (2026-06-22T03:48:56.128Z)

The capabilities endpoint exists on this engine branch at `backend/main.py:173`. Let me read that endpoint to see exactly how `default\_segmenter` and `gemini\_key\_present` are computed and whether they can be inconsistent.

20. user (2026-06-22T03:48:56.650Z)

```

150     keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
151     """
152     if segments:
153         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
154     else:
155         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
156
157

```

```

158 # --- API Endpoints ---
159 @app.get("/api/config")
160 def get_config():
161     return global_config
162
163 @app.get("/api/health")
164 def health():
165     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
166     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
167     confirm the engine is reachable and which detector is wired. Never raises."""
168     sport = getattr(global_config, "sport", None)
169     indexing = getattr(global_config, "indexing", None)
170     return {
171         "status": "ok",
172         "sport": sport,
173         "default_segmenter": getattr(indexing, "default_segmenter", None),
174     }
175
176 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
177     """The full hash ? validate ? segment ? report pipeline for one video.
178
179     This is the former synchronous /api/process body, unchanged in behavior, now run on
180     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
181     the sync endpoint used to return (UI compatibility); the per-video observability
182     report is still emitted in `finally:` and grafted as response["reports"].
183
184     `progress` is an optional callable(stage: str) used to surface coarse job progress.
185     Raises on unexpected internal errors ? the caller records those as a job failure
186     (after this function has already restored the pre-run segments, see below).
187     """
188     notify = progress or (lambda stage: None)
189
190     notify("hashing")
191     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
192     # (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
193     # original # req.video_path (the source URI) is kept for the DB record + report; only the file-
194     # reading # consumers (hash / validators / segmenter) use the resolved local path.
195     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
196     "storage", None))
197     video_id = compute_video_id(local_video)
198     was_reingest = db_instance.get_video(video_id) is not None
199     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
200     report)
201     notify("validating")
202     logger.info(f"Running validations for {req.video_path}...")
203     val_results, val_context = registry.run_all_with_context(local_video, global_config)
204     failures = [r for r in val_results if not r.passed]
205
206     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
207     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
208     "motion")
209     seg_name = req.segmenter or default_seg
210     segmenter_actual = None
211     segmentation_ran = False
212     response: Optional[Dict[str, Any]] = None
213     try:
214         if failures:
215             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
216             # status; it no longer destroys the video's prior good segments.
217             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
218             for r in val_results])

```

```

216         response = {
217             "video_id": video_id,
218             "status": "failed",
219             "message": "Video failed validation checks.",
220             "results": [r.model_dump() for r in val_results],
221         }
222         return response
223
224     # 2. Run segmenter & indexer
225     logger.info("[API] Video passed checks. Segmenting and indexing...")
226     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
227
228     segmenter_cls = segmenter_registry.get(seg_name)
229     if not segmenter_cls:
230         # Submit-time validation already 400s unknown names; this is the defensive
231         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
232         available = list(segmenter_registry.list_available().keys())
233         response = {
234             "video_id": video_id,
235             "status": "failed",
236             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
237             "results": [r.model_dump() for r in val_results],
238             "play_segments": [],
239         }

```

21. assistant (2026-06-22T03:48:58.947Z)

I see `/api/health` but the finding is about `/api/capabilities`. Let me find the actual capabilities endpoint that returns `gemini\_key\_present`.

22. user (2026-06-22T03:49:00.869Z)

```

backend\reporting\harvest.py:153:         "api_key_present": bool(os.environ.get(key_env)) if
ai_based else None,

```

23. assistant (2026-06-22T03:49:08.325Z)

The `/api/capabilities` endpoint does not exist in the engine yet (only `/api/health` returns `default\_segmenter`, and there's no `gemini\_key\_present` field anywhere). This is consistent with the wizard's comment that it's built against an "honest probe" contract that the engine is meant to provide. The wizard is defensively designed against a not-yet-shipped contract.

Now the key question for the finding: **regardless of whether the endpoint exists yet, can the contract legitimately produce `{gemini\_key\_present: false, default\_segmenter: "gemini"}`?**

Let me reason about this carefully. The `default\_segmenter` is a config value (`indexing.default\_segmenter`), which is independent of whether a Gemini key is present. Looking at `\_execute\_processing` at line 206-207, the engine uses `default\_seg` directly without checking key presence? so if config says `default\_segmenter: "gemini"` but no key is set, the engine would attempt gemini and likely fail at runtime (no automatic fallback to motion in the resolution path I can see). So the two fields ARE independent and the contradictory state is representable.

Let me confirm there's no fallback that rewrites `default\_segmenter` to motion when the key is absent.

24. user (2026-06-22T03:49:09.836Z)

```

backend/eval/gemini_refine.py:17:Run (GEMINI_API_KEY in env):
backend/eval/gemini_refine.py:105:     api_key = os.environ.get("GEMINI_API_KEY")
backend/eval/gemini_refine.py:107:         raise SystemExit("GEMINI_API_KEY not set in env")

```

```

backend/eval/gemini_refine.py:156:     api_key = os.environ.get("GEMINI_API_KEY")
backend/eval/gemini_refine.py:158:         raise SystemExit("GEMINI_API_KEY not set in env")
backend/utils/app_home.py:62:     """`<app-home>/env` ? the dotenv file the server prefers
(e.g. ``GEMINI_API_KEY``)."""

```

25. assistant (2026-06-22T03:49:14.470Z)

There's no engine-side logic that automatically rewrites `default\_segmenter` to "motion" when the Gemini key is absent. The two fields are genuinely independent in the contract. So the contradictory state `{gemini\_key\_present: false, default\_segmenter: "gemini"}` is representable ? it would occur on a misconfigured box where config sets gemini as default but no key is readable. In that case, KeyStep would say "works offline with motion detector" while ComputeStep says "Detector: gemini" ? a real contradiction in the wizard's surfaced text.

Now let me assess severity honestly. Let me verify the exact i18n strings to confirm the claimed contradiction wording.

26. user (2026-06-22T03:49:16.103Z)

```

{'compute': {'detector': 'Detector: {name}',
  'ffmpegMissing': 'ffmpeg not found ? install it so KhelSutra can '
    'cut and join clips.',
  'ffmpegOk': 'Video tools (ffmpeg) ready.',
  'gpu': 'A GPU was detected ? the high-quality shuttle tracker can '
    'run here later.',
  'local': 'Everything runs on this device ? your videos never '
    'leave it.',
  'noGpu': 'No GPU needed ? the offline motion detector runs on any '
    'computer.',
  'title': 'Where it runs'},
'key': {'connected': "Gemini is connected ? you'll get AI-assisted "
  'highlights.',
  'getKey': 'Get a free Gemini key',
  'how': 'Want sharper highlights? Get a free Gemini key, add '
    'GEMINI_API_KEY=? to the .env file in your KhelSutra folder, '
    'then restart.',
  'missing': "No AI key yet ? that's fine. KhelSutra works offline with "
    'the built-in motion detector.',
  'title': 'AI highlights (optional)'}}

```

27. assistant (2026-06-22T03:49:26.825Z)

The strings confirm the finding's quoted text. In the contradictory state:

- Step 1 (KeyStep, key absent): "No AI key yet ? that's fine. KhelSutra works offline with the built-in motion detector."
- Step 2 (ComputeStep): "Detector: gemini"

That is a genuine contradiction surfaced verbatim from two independent fields.

Now the adversarial honesty check on severity and likelihood:

1. ****Is the state reachable?*** Yes, the contract permits it (fields are independent), and the engine has no auto-fallback rewriting `default\_segmenter` to motion when the key is missing.
2. ****How likely / how harmful?*** This is a misconfigured-box edge case (gemini configured as default but key unreadable). In practice, the engine's own default is `yolo\_hybrid` (config) or `motion` (the wizard test LIGHT tier), and the `/api/capabilities` endpoint isn't even shipped yet ? the wizard is written against a contract. On the LIGHT bundle north-star, `default\_segmenter` is "motion" and the key is absent, which is consistent. The contradiction only surfaces on a specific misconfiguration. It's cosmetic (confusing onboarding text), not a functional bug ? no crash, no data issue. The finding itself self-describes as "low-risk."
3. ****Is the fix appropriate?*** The suggested fix (caveat in ComputeStep when `default\_segmenter=="gemini" && gemini\_key\_present!=true`) is reasonable and low-cost.

This holds up as a ****real but minor**** consistency gap. It is correctly classified as a nit: it's a UI text contradiction in a reachable-but-unlikely misconfigured state, cosmetic, no functional impact, and the author explicitly built the wizard to surface engine state "verbatim" without reconciliation. The claimed severity of "nit" is accurate.

Let me do one final check ? confirm there isn't already reconciliation logic elsewhere (e.g., App.tsx) that would prevent the contradiction, which would make this a false alarm.

28. user (2026-06-22T03:49:28.294Z)

```
diff --git a/web/src/App.tsx b/web/src/App.tsx
index a0a7882..c516153 100644
--- a/web/src/App.tsx
+++ b/web/src/App.tsx
@@ -8,6 +8,7 @@ import { SelectionFooter } from "../components/SelectionFooter";
import { LanguageSwitcher } from "../components/LanguageSwitcher";
import { CompileResult, type BuildState } from "../components/CompileResult";
import { IngestPanel } from "../components/IngestPanel";
+import { SetupWizard, isWizardDone } from "../components/SetupWizard";
import { runQuery, type ParsedQuery } from "../lib/query";
import { floorRisk } from "../lib/floor";
import { reelFilename } from "../lib/format";
@@ -60,6 +61,8 @@ export default function App() {
  const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" } : {
status: "demo" });
  const [reelName, setReelName] = useState("highlights");
  const [build, setBuild] = useState<BuildState>({ status: "idle" });
+ // P2c: the first-run wizard shows once per browser, before the ingest panel, when nothing's
loaded.
+ const [wizardDone, setWizardDone] = useState(isWizardDone);

  // Display order is separate from selection: a query can re-sort the cards while the shared
  // selection set (sel) stays the single source of truth for what lands in the reel.
@@ -234,9 +237,13 @@ export default function App() {
  <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
</p>

+    /* P2c first-run wizard: before anything is loaded, walk a new user through AI-key /
compute /
+    videos (from /api/capabilities). Shows once per browser; then the ingest panel
takes over. */
+    {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
+
    /* Front-half ingest (B-P2): when no video is loaded, point the tool at a bucket URI /
host
    path ? process ? load. Once a video_id exists, the existing back-half takes over.
*/
-    {!videoId && (
+    {!videoId && wizardDone && (
      <IngestPanel
        onLoad={() => {
          setVideoId(id);
diff --git a/web/src/api/client.ts b/web/src/api/client.ts
index fa74d2f..23c8dd4 100644
--- a/web/src/api/client.ts
+++ b/web/src/api/client.ts
@@ -7,6 +7,7 @@ import type {
  QueryResponse,
  CompileResponse,
  EngineConfig,
+ EngineCapabilities,
  ProcessResponse,
  JobStatus,
  UploadInitResponse,
@@ -133,6 +134,11 @@ export async function getConfig(cid?: string): Promise<EngineConfig> {
```

```

    return request<EngineConfig>("/config", { method: "GET", cid });
}

+/** What THIS box can do ? drives the first-run wizard. GET /api/capabilities (PACKAGING_PLAN
P2c). */
+export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
+  return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
+}
+
+  /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
  export function minShotCount(cfg: EngineConfig): number | undefined {
    return cfg.stitching?.min_shot_count;

```

29. assistant (2026-06-22T03:49:39.940Z)

No reconciliation logic exists anywhere ? `App.tsx` just renders `SetupWizard`, and `client.ts` is a thin passthrough. The two wizard steps each render their respective field independently with no cross-check. The finding holds up.

Verification complete. My assessment:

****The finding is REAL but a true nit.****

- The cited code is accurate: `KeyStep` (SetupWizard.tsx:26) keys off `caps?.gemini\_key\_present === true` and `ComputeStep` (line 54) independently renders `caps?.default\_segmenter ?? "motion"`. No reconciliation exists between them (confirmed in App.tsx and client.ts too).
- The contradictory contract state `{gemini\_key\_present: false, default\_segmenter: "gemini"}` is genuinely representable: in the engine, `default\_segmenter` is a config value (`backend/config/models.py:128`, `backend/main.py:206-207`) fully independent of key presence, and there is no engine-side fallback that rewrites it to "motion" when no key is set. So a misconfigured box can yield Step 1 saying "works offline with the built-in motion detector" while Step 2 says "Detector: gemini" ? verified against the actual en strings.
- But it is correctly a nit: cosmetic onboarding-text contradiction only, no functional impact, reachable only on a specific misconfiguration (gemini set as default but key unreadable). On the intended LIGHT bundle the state is consistent (motion + no key), and the `/api/capabilities` endpoint the wizard consumes isn't even shipped in the engine yet (only `/api/health` exists). The author deliberately surfaces engine state verbatim. The suggested ComputeStep caveat is a reasonable optional polish.

30. user (2026-06-22T03:49:46.113Z)

Structured output provided successfully

31. assistant (2026-06-22T03:49:51.356Z)

Verification complete. The finding is REAL but correctly classified as a nit ? a cosmetic onboarding-text contradiction that is reachable only on a specific misconfigured-box state, with no functional impact.

Relevant files:

- `C:\Users\avidu\Projects\khelsutra-p2c\web\src\components\SetupWizard.tsx` (lines 26, 54 ? the two independent field reads)
- `C:\Users\avidu\Projects\khelsutra-p2c\web\src\api\types.ts` (lines 54-59 ? the independent `default\_segmenter` / `gemini\_key\_present` contract fields)
- `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:128` and `backend/main.py:206-207` (engine uses `default\_segmenter` config value with no key-based fallback to motion, confirming the contradictory state is representable)